

9. 텍스트 분류 (2)

goblurry · 18분 전

통계 수정 삭제

데이터분석

머신러닝

파완머



파이썬 머신러닝 완벽가이드

▼ 목록 보기

10/10



 파이썬 머신러닝 완벽 가이드 (위키북스, 개정 2판) 교재의 8장을 바탕으로 학습한 내용을 정리한 포스트입니다.

목차

1. 토픽 모델링
2. 문서 군집화 소개와 실습
3. 문서 유사도
4. 한글 텍스트 처리

토픽 모델링

토픽 모델링이란 문서 집합에 숨어 있는 주제를 찾아내는 기법이다. 문서의 양이 많아질수록 사람이 직접 모든 문서를 읽고 핵심 주제를 파악하는 데에는 한계가 있다. 이때 머신러닝 기반의 토픽 모델링을 적용하면, 대량의 문서에서 중심 주제를 효과적으로 추출할 수 있다.

사람이 수행하는 요약은 문장을 압축해 의미를 전달하는 데 초점이 맞춰져 있다면, 머신러닝 기반의 토픽 모델링은 문서 전체에서 주제를 대표하는 핵심 단어들의 집합을 추출해 숨겨진 주제를 표현한다는 점에서 차이가 있다.

머신러닝 기반 토픽 모델링 기법

머신러닝 기반 토픽 모델링에서 가장 많이 사용되는 기법은 LSA(Latent Semantic Analysis) 와 LDA(Latent Dirichlet Allocation) 이다.

이 중 이 절에서는 LDA 기반 토픽 모델링을 중심으로 다룬다.

*주의할 점은, 토픽 모델링에서 사용하는 LDA(Latent Dirichlet Allocation) 는 차원 축소 기법인 LDA(Linear Discriminant Analysis) 와 이름만 같고 완전히 다른 알고리즘이라는 것이다.

LDA 기반 토픽 모델링 개요

LDA는 문서 집합에서 각 문서가 여러 토픽의 확률 분포로 구성되어 있다는 가정을 기반으로 한다. 즉, 하나의 문서는 하나의 주제만 가지는 것이 아니라, 여러 토픽이 서로 다른 비중으로 섞여 있는 구조라고 본다.

LDA 모델은 다음 두 가지를 동시에 학습한다.

- 각 토픽을 구성하는 주요 단어 분포
 - 각 문서가 어떤 토픽들로 구성되어 있는지에 대한 분포
- 이를 통해 문서 전체에 숨어 있는 주제를 추론한다.

LDA와 피처 벡터화 방식

LDA는 Count 기반의 벡터화 결과만을 입력으로 사용한다. 즉, TF-IDF 기반 벡터화는 LDA에 사용할 수 없다. 이는 LDA가 단어의 출현 빈도를 기반으로 확률 분포를 추정하는 모델이기 때문이다.

따라서 토픽 모델링을 적용할 경우, 텍스트는 반드시 CountVectorizer를 이용해 벡터화해야 한다. 또한 단어 피처 수가 지나치게 많아질 경우 토픽 해석이 어려워질 수 있으므로, 피처 개수 제한이나 n-gram 범위 설정 등을 통해 적절히 조정한다.

LDA 모델의 핵심 결과: components_

LDA 모델을 학습하면 components_ 속성을 가지게 된다. 이 값은 각 토픽별로 단어 피처가 얼마나 강하게 연관되어 있는지를 나타내는 행렬이다.

행(row): 토픽

열(column): 단어 피처

값(value): 해당 단어가 특정 토픽에 할당된 정도

값이 클수록 해당 단어는 그 토픽을 대표하는 핵심 단어라고 볼 수 있다. components_의 형태는 (토픽 개수 × 단어 피처 개수) 로 구성된다.

토픽 해석과 한계

LDA는 수치 결과만 제공하므로, 각 토픽이 무엇을 의미하는지는 사람이 직접 해석해야 한다. 이를 위해 일반적으로 각 토픽에서 연관도가 높은 단어들을 추출해 나열하고, 그 단어들의 조합을 바탕으로 토픽의 의미를 판단한다. 실제로 일부 토픽은 명확한 주제가 추출되지만, 일반적인 단어가 주를 이루거나 여러 영역의 단어가 섞여

해석이 모호한 토픽이 생성되는 경우도 많다. 특히 주제 간 경계가 불명확하거나, 데이터 자체의 품질이 낮은 경우에는 토픽 모델링의 성능이 제한적일 수 있다.

정리

LDA 기반 토픽 모델링은 대규모 문서 집합에서 숨겨진 주제를 탐색하는 데 효과적인 비지도 학습 기법이다. 다만 결과 해석이 자동화되지 않으며, 토픽의 명확성은 데이터 특성과 전처리, 피처 설정에 크게 의존한다는 점을 함께 고려해야 한다.

문서 군집화

문서 군집화(Document Clustering)는 텍스트 구성이 유사한 문서들을 자동으로 묶는 기법이다.

문서 군집화는 동일한 군집에 속한 문서를 같은 카테고리로 분류할 수 있다는 점에서 텍스트 분류와 유사하지만,

사전에 정답 레이블이 필요하지 않은 비지도 학습이라는 점에서 차이가 있다.

즉, 텍스트 분류가 “어떤 카테고리에 속하는지 맞는 문제”라면, 문서 군집화는 “비슷한 문서끼리 자연스럽게 묶는 문제”라고 볼 수 있다.

문서 군집화와 텍스트 분류의 차이

텍스트 분류

1. 사전에 정의된 카테고리 필요
2. 레이블이 있는 학습 데이터 필요
3. 지도 학습 기반

문서 군집화

1. 카테고리 사전 정의 불필요
2. 학습 데이터에 레이블 없음
3. 비지도 학습 기반

이 장에서는 기존에 배운 군집화 알고리즘을 텍스트 데이터에 적용해 문서 군집화를 수행한다.

군집 수(K)에 따른 군집화 특성

군집화에서 군집 개수(K)는 결과에 큰 영향을 미친다.

군집 수를 크게 설정하면

→ 더 세분화된 군집이 생성되지만, 주제 해석이 어려워질 수 있다.

군집 수를 줄이면

→ 군집은 보다 큰 범주로 묶이지만, 서로 다른 주제가 하나의 군집에 포함될 가능성이 있다.

문서 군집화에서는 데이터의 특성과 분석 목적에 따라 적절한 군집 수를 선택하는 것이 중요하다.

군집별 핵심 단어 추출의 필요성

군집 번호만으로는 각 군집이 어떤 주제를 나타내는지 알기 어렵다. 이를 해석하기 위해 각 군집을 대표하는 핵심 단어를 추출한다.

군집화 알고리즘은 각 **군집의 중심점(Centroid)**을 기준으로 문서를 할당한다. 이 중심점은 해당 군집을 대표하는 피쳐들의 상대적 중요도 정보를 포함하고 있다.

즉, 군집 중심에서 값이 큰 단어일수록 그 군집을 설명하는 핵심 단어라고 볼 수 있다.

정리

문서 군집화는 레이블이 없는 대규모 텍스트 데이터에서

문서 간 유사성을 기반으로 주제별 구조를 파악할 수 있는 효과적인 비지도 학습 기법이다.

다만, (1) 군집 개수 설정 (2) 피쳐 벡터화 방식 (3) 데이터 품질에 따라 결과가 크게 달라질 수 있으며, 군집 결과는 항상 사람의 해석을 통해 검증되어야 한다는 점을 함께 고려해야 한다.

문서 유사도

문서 유사도는 두 문서가 얼마나 비슷한 내용을 담고 있는지를 수치로 나타내는 개념이다. 텍스트 분석에서는 문서를 벡터로 변환한 뒤, 벡터 간 유사도를 계산하는 방식으로 문서 유사도를 측정한다.

코사인 유사도(Cosine Similarity)

문서 유사도 측정에 가장 널리 사용되는 방법은 코사인 유사도이다.

코사인 유사도는 두 벡터의 크기(length)가 아니라, 방향(direction)이 얼마나 유사한지를 기준으로 유사도를 계산한다. 즉, 두 문서 벡터가 이루는 사잇각의 코사인 값을 통해 문서 간 유사도를 판단한다.

$$A * B = \|A\| \|B\| \cos \theta$$

따라서 유사도 $\cos \theta$ 는 다음과 같이 두 벡터의 내적을 총 벡터 크기의 합으로 나눈 것입니다(즉, 내적 결과를 총 벡터 크기로 정규화(L2 Norm)한 것입니다).

$$\text{similarity} = \cos \theta = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

값의 범위는 -1 ~ 1

- 1에 가까울수록 매우 유사
- 0에 가까우면 관련성 없음
- 음수는 반대 방향의 관계를 의미

텍스트 분석에서는 보통 0~1 사이 값만 등장한다.

코사인 유사도를 사용하는 이유

코사인 유사도가 문서 유사도 측정에 적합한 이유는 다음과 같다.

1. **문서 길이의 영향을 줄일 수 있음**: 긴 문서는 단어 수가 많아 벡터 크기가 커지기 쉽다. 코사인 유사도는 벡터를 정규화(L2 Norm)하기 때문에 문서 길이가 달라도 공정한 비교가 가능하다.
2. **고차원·희소 벡터에 적합**: TF-IDF로 변환된 문서 벡터는 차원이 크고 대부분 값이 0인 희소 행렬이 된다. 이런 환경에서는 거리 기반 지표보다 코사인 유사도가 안정적이다.
3. **단어 빈도 편향 완화**: 특정 단어가 많이 등장한다고 해서 무조건 더 관련 있는 문서라고 판단하지 않는다. 문서 전체 맥락에서의 방향성을 기준으로 비교한다.

문서 유사도 계산 흐름

문서 유사도 측정의 전체 흐름은 다음과 같다.

1. 문서를 TF-IDF 벡터로 변환
2. 각 문서를 고차원 벡터로 표현
3. 벡터 간 코사인 유사도 계산
4. 유사도 값이 높은 문서일수록 내용이 유사하다고 판단

한글 텍스트 처리와 감성 분석

앞 절까지는 영어 기반 텍스트 분석을 다뤘다면, 이번 절에서는 한글 텍스트 처리의 특성과 이를 실제 감성 분석에 어떻게 적용하는지를 살펴본다. 한글은 영어와 달리 띄어쓰기 규칙이 엄격하지 않고 조사와 어미가 단어에 결합되는 구조를 가지기 때문에, 단순히 공백 기준으로 단어를 분리하는 방식만으로는 의미 있는 분석이 어렵다. 이로 인해 한글 텍스트 분석에서는 형태소 분석을 기반으로 한 전처리가 중요한 역할을 한다.

한글 NLP 처리의 어려움

일반적으로 한글 언어 처리는 영어 등 라틴어 계열 언어보다 복잡하다. 가장 큰 이유는 띄어쓰기와 조사 때문이다. 띄어쓰기를 잘못하면 의미 자체가 달라질 수 있으며, 동일한 단어라도 조사나 어미에 따라 매우 다양한 형태로 등장한다. 예를 들어 '집'이라는 단어는 '집에', '집에서', '집으로' 등 여러 형태로 표현되며, 이를 하나의 개념으로 묶지 않으면 단어 빈도가 분산되는 문제가 발생한다.

또한 조사 형태가 단어의 일부인지, 의미를 가지는 명사인지 구분하기 어려운 경우도 많아 어근 추출이나 정규화 과정이 영어보다 까다롭다. 이러한 특성 때문에 한글 텍스트는 단어 단위보다는 **형태소 단위**로 분석하는 것이 보다 안정적인 결과를 얻을 수 있다.

KoNLPy 소개

KoNLPy: 파이썬 환경에서 사용할 수 있는 대표적인 한글 형태소 분석 패키지

형태소 분석: 문장을 의미를 가지는 최소 단위인 형태소로 분해하고, 각 형태소에 품사 정보를 부여하는 과정

KoNLPy를 사용하면 한글 문장을 **형태소 단위의 토큰 리스트**로 변환할 수 있으며, 이를 통해 조사나 어미를 분리한 의미 중심의 단어 분석이 가능해진다.

이러한 형태소 기반 토큰화 결과는 이후 TF-IDF 벡터화나 분류 모델과 자연스럽게 연결할 수 있어, 한글 텍스트를 머신러닝 모델이 처리할 수 있는 형태로 변환하는 핵심 단계라고 볼 수 있다.

감성 분석 실습의 목적

이 장에서는 네이버 영화 평점 데이터(<https://github.com/e9t/nsmc>)를 이용해 한글 텍스트 감성 분석을 수행한다. 리뷰 문장을 형태소 분석을 통해 토큰화한 뒤, 이를 벡터화하여 긍정과 부정을 분류하는 과정을 다룬다. 구현 자체보다는 한글 텍스트가 어떤 전처리 과정을 거쳐 수치 데이터로 변환되고, 그 결과가 분류 모델에 어떻게 활용되는지를 이해하는 데 목적이 있다.

이 과정을 통해 한글 텍스트 분석에서 형태소 분석이 왜 중요한지, 그리고 전처리 방식이 모델 성능에 얼마나 큰 영향을 미치는지를 확인할 수 있다. 또한 영어 텍스트 분석에서 사용하던 일반적인 머신러닝 분류 기법이, 적절한 한글 전처리를 거친 경우에도 충분히 적용 가능하다는 점을 이해할 수 있다. 결국 이번 실습은 한글 텍스트를 데이터로 다루는 전체 흐름을 개념적으로 정리하는 데 의미가 있다.

정리

8.6~8.9절에서는 문서를 수치 벡터로 표현한 뒤 문서 간 의미적 유사도를 측정하는 방법을 다루며, 그 핵심으로 코사인 유사도를 사용한다.

코사인 유사도는 문서 벡터의 크기보다 방향성에 집중해 두 문서가 얼마나 비슷한 주제를 다루는지를 측정하는 지표로, TF-IDF처럼 고차원·희소 벡터 환경에서 특히 효과적이다. 이를 통해 문서 쌍 간 유사도 계산, 기준 문서와 다른 문서들의 유사도 비교, 전체 문서 집합에 대한 유사도 행렬 생성이 가능하며, 단순 빈도 기반 비교의 한계를 보완할 수 있다. 또한 군집화된 문서 집합 내부에서 유사도를 활용하면 특정 문서와 가장 유사한 문서를 찾거나, 군집의 주제적 일관성을 해석하는 데 활용할 수 있음을 보여준다.



goblurry



이전 포스트

8. 텍스트 분석

0개의 댓글

댓글을 작성하세요

댓글 작성



Powered by
Stellate